

TRUST & SAFETY CLASSIFICATION MODEL

Development Report — From Raw Data to a Stabilized BiLSTM Architecture

Overview

This report traces the iterative development of a text classification model for Trust & Safety moderation, covering query and image-description data across nine categories (e.g., safe, violent crimes, non-violent crimes, elections, unsafe, unknown s-type, and several minority classes). The work progressed through three connected phases: data engineering, architecture evolution from a Bidirectional RNN to a Bidirectional LSTM, and a series of technical debugging cycles that were necessary to reach stable training. Each phase built directly on lessons learned from the one before it.

1. Data Engineering Phase

1.1 From Horizontal Concatenation to Vertical Stacking

The raw dataset stored each row as a query, an image description, and a shared Toxic Category label side by side in the same row — effectively a horizontal layout where two different text modalities lived in separate columns. Early attempts treated these columns independently, which risked misaligning samples and under-using the image-description text as training signal in its own right.

The pipeline was restructured to use vertical stacking instead: the query column and the image-description column were each split out into their own text/label pairs and then concatenated row-wise (`pd.concat` with the columns renamed to a common text field) rather than joined side by side. This ensured every text sample — whether a search query or an image caption — carried its own row and its own aligned label, doubling the effective number of training examples and keeping the query and image-description streams consistent with one another.

1.2 Duplicate Removal

Once the data was stacked vertically, a large number of exact duplicate rows surfaced — the same generic image captions (e.g., a family enjoying a picnic in the park) and repeated boilerplate queries appeared dozens of times across the safe and majority classes. Left unaddressed, this redundancy caused the model to memorize specific recurring strings rather than learn generalizable patterns, inflating training accuracy while hurting real-world generalization.

A strict `drop_duplicates()` pass was applied immediately after stacking (and again after merging in synthetic data later in the pipeline). This step alone reduced the working dataset from several thousand rows down to a much smaller, de-duplicated core — confirming how much of the original volume had been redundant rather than genuinely diverse.

1.3 Data Quality Problems Uncovered

De-duplication exposed several underlying data-quality issues that needed direct correction:

- **Mislabeled rows:** manual spot-checks found individual rows with incorrect category assignments (e.g., a real-estate investment query mislabeled as ‘violent crimes’, or a generic health question mislabeled as ‘elections’). These were corrected by index once identified.
- **Noise strings:** at least one templated placeholder query (“What are the health benefits of daily exercise...”) had been stamped across multiple categories as filler text and had to be filtered out entirely rather than corrected, since it carried no genuine signal for any class.
- **Inconsistent casing and whitespace:** text fields were stripped and lower-cased, and empty strings were converted to NaN so they wouldn’t silently masquerade as valid data.
- **Static image-description bank:** because image descriptions were drawn from a small fixed set of caption templates, a manual correction dictionary (`image_corrections`) was built to map each recurring caption to its correct category, since the same caption could otherwise be inconsistently labeled across rows.

1.4 Addressing Class Imbalance

Even after cleaning, the category distribution was severely imbalanced: majority classes such as ‘safe’ and ‘violent crimes’ numbered in the hundreds, while minority classes such as ‘elections’, ‘sex-related crimes’, ‘suicide & self-harm’, and ‘child sexual exploitation’ had as few as five examples each. Two complementary strategies were used to address this:

- **Synthetic data & augmentation:** additional synthetic samples were generated and merged into the training set specifically to bolster underrepresented classes (elections, unknown s-type, non-violent crimes, sex-related crimes, and related minority categories). Synthetic contributions were deliberately filtered to exclude the already well-represented ‘violent crimes’ class, so augmentation targeted only the classes that actually needed it rather than further inflating an already-dominant category.
- **Algorithmic weighting via Focal Loss:** alongside data-level augmentation, the training loss itself was changed from standard cross-entropy to Focal Loss. Focal Loss down-weights the contribution of easy, already-well-classified examples (typically from majority classes) and up-weights harder, misclassified examples — combined with class weights derived from `compute_class_weight`, this shifted the model’s learning focus away from simply predicting the majority class by default and toward correctly distinguishing the rarer, harder categories.

Together, these two approaches attacked the imbalance problem from both directions — more representative data for rare classes, and a loss function that penalizes the model more heavily for getting those rare classes wrong.

2. Architecture Evolution: RNN vs. LSTM

A central challenge throughout the project was the model’s inability to retain context across a full sentence, which showed up as poor classification on nuanced, context-dependent content.

2.1 The RNN Bottleneck

The initial approach used a Bidirectional RNN (ToxicRNN). It suffered from two related problems: vanishing gradients, and what can be described as ‘padding washout.’ Because every sequence was padded to a fixed maximum length (45 tokens in the tokenizer-based version, 40 in the later TextVectorization-based version), the model was forced to process the full padded length even for very short sentences. The RNN’s internal hidden state faded across these extra steps, and by the time the network reached the end of a padded sequence, the signal from meaningful tokens near the beginning of the text had often been washed out.

2.2 The LSTM Pivot

To address this, the architecture was upgraded to a Bidirectional LSTM (BiLSTM). Unlike a vanilla RNN, the LSTM maintains an internal cell state — effectively a protected ‘conveyor belt’ of information that flows through the sequence largely unmodified except where its gates deliberately choose to write or forget. This made the model far more resistant to having its signal overwritten by long runs of padding zeros.

2.3 Packing

Two further refinements were layered onto the LSTM to fully solve the padding problem:

- **pack_padded_sequence:** sequence lengths (derived from the attention mask / true token counts) were passed into `pack_padded_sequence` so that the RNN/LSTM layers skip over padding tokens entirely during the forward pass, rather than processing them as if they were meaningful input.

3. Technical Challenges & Debugging

Reaching stable training required resolving several recurring technical issues:

3.1 Data Quality vs. Embeddings

Pre-trained GloVe embeddings (trained circa 2014) failed to recognize modern platform-specific vocabulary such as ‘deepfake’ and ‘onlyfans,’ leaving those tokens without a meaningful pre-trained vector. A regex-based text-cleaning pipeline was built to close some of this gap by: normalizing smart quotes and em/en dashes into plain characters, fixing smashed-together contractions (e.g., turning malformed tokens like ‘what’swerent’ into properly spaced words), and expanding contractions (n’t → not, ’re → are, ’s → is, ’ll → will, ’ve → have, ’m ’ am) before stripping any remaining non-alphanumeric characters.

4. Final Findings & Performance Comparison

The transition from a simple BiRNN to a context-aware BiLSTM — paired with a cleaner, better-balanced dataset and Focal Loss — produced a clear improvement in both F1-score and generalization, achieved in fewer training epochs:

Metric	BiRNN Implementation	BiLSTM Implementation
Epochs	25	25
Final F1-Score (macro)	0.5807	0.6605
Accuracy	56%	81.96%

Key Takeaways

- **Context is king:** the LSTM's protected cell state made it substantially better at handling ambiguous, 'middle ground' categories — such as unsafe and non-violent crimes — where correct classification depends on retaining context across the whole sentence rather than just spotting a single keyword..
- **Stability:** moving to `pack_padded_sequence` eliminated padding noise from the training signal, producing a cleaner loss curve and more reliable, repeatable convergence across runs.

End of report.